

**INSTITUTO
FEDERAL**
Santa Catarina

Programação Funcional em JavaScript (parte 1)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

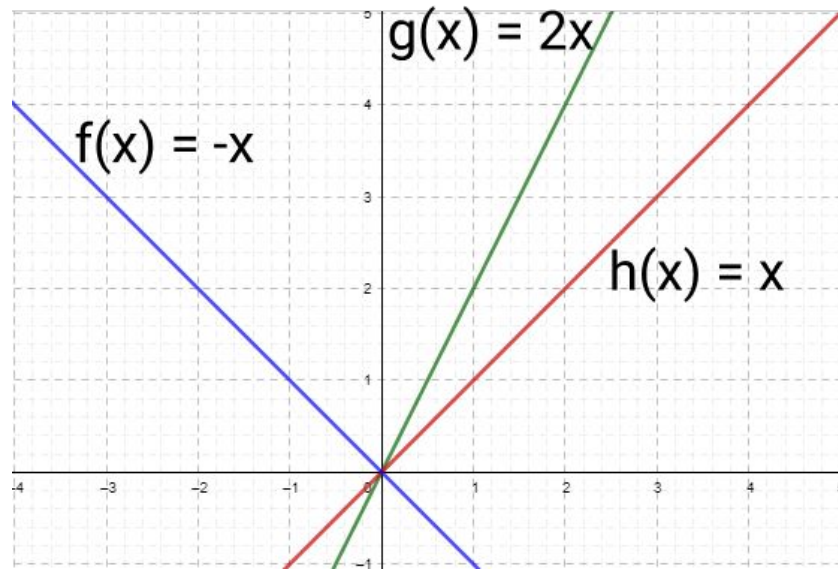
Roboto Mono



Programação Funcional

o que é

- computação como a avaliação de funções matemáticas
- prioriza a declaração do **o que fazer** em vez de como fazer
- foco em
 - funções puras
 - imutabilidade
 - composição de funções



Programação Funcional

vantagens

- previsibilidade e facilidade de depuração
- imutabilidade
- modularidade e reutilização
- concorrência

desafios

- curva de aprendizado
- desempenho
- dificuldade com efeitos colaterais



Funções Puras

características principais

- **determinismo**
 - mesmas entradas, mesmos resultados
- **sem efeitos colaterais**
 - não altera nenhum estado fora do seu escopo

exemplos

```
let multiplicador = 2;

// função impura
function dobrarImpuro(numero) {
  return numero * multiplicador;
}

// função pura
function dobrarPuro(numero) {
  return numero * 2;
}
```



Funções Puras

características principais

■ determinismo

- mesmas entradas, mesmos resultados

■ sem efeitos colaterais

- não altera nenhum estado fora do seu escopo

exemplos

```
const numeros = [4, 3, 2, 1];
```

```
const ordenados = numeros.sort( (a, b) => a - b );  
// a função sort() é sempre impura!  
// ela modifica o array original
```

```
const dobro = numeros.map( n => n * 2 );  
// aqui, map() é pura  
// não modifica o array original,  
// nem depende ou altera um estado externo
```

```
let multiplicador = 2;  
const dobro = numeros.map( n => n * multiplicador );  
// aqui, map() é impura  
// depende de um estado externo
```



Funções Puras

características principais

■ determinismo

- mesmas entradas, mesmos resultados

■ sem efeitos colaterais

- não altera nenhum estado fora do seu escopo

exemplos

```
// somar os elementos
```

```
let numeros = [4, 3, 2, 1];
```

```
// programação imperativa
```

```
let soma = 0;
```

```
for (let i = 0; i < numeros.length; i++) {  
  soma += numeros[i];  
}
```

```
console.log(soma);
```

```
// programação funcional
```

```
let soma = numeros.reduce( (acc, n) => acc + n, 0 );
```

```
console.log(soma);
```



Funções de Ordem Superior

o que são

- **funções que operam sobre funções**
 - funções como argumentos

exemplos

```
const executar = (a, b, operacao) => operacao(a, b);
```

```
const somar = (x, y) => x + y;  
const multiplicar = (x, y) => x * y;
```

```
executar(5, 10, somar);  
executar(5, 10, multiplicar);
```



Funções de Ordem Superior

o que são

- **funções que operam sobre funções**
 - funções como argumentos

exemplos

```
// encontrar os pares
const numeros = [4, 3, 2, 1];

// programação imperativa
const pares = [];

for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    pares.push(numeros[i]);
  }
}

// programação funcional
const ehPar = numero => numero % 2 === 0;

const pares = numeros.filter(ehPar);

// alternativa mais direta
const pares = numeros.filter( x => x % 2 === 0 );
```



Funções de Ordem Superior

o que são

- **funções que operam sobre funções**
 - funções como argumentos
 - retornam funções

exemplos

```
const somar = a => b => a + b;  
// const somar = a => { return b => a + b };
```

```
const somarCinco = somar(5);  
// const somarCinco = b => 5 + b;
```

```
console.log(somarCinco(10)); // 15
```

```
const somarDez = somar(10);  
// const somarDez = b => 10 + b;
```

```
console.log(somarDez(7)); // 17
```



Funções de Ordem Superior

o que são

- **funções que operam sobre funções**
 - funções como argumentos
 - retornam funções
- **funções são cidadãos de 1ª classe**

exemplos

```
let senha = 'minhasenha';
let nomeUsuario = 'jose';

// validação imperativa
const validar = (valor, min, max) => {
  return valor.length >= min && valor.length <= max;
}
validar(senha, 8, 16);
validar(nomeUsuario, 4, 12);

// validação funcional
const validar = (min, max) => {
  return (valor) => valor.length >= min &&
    valor.length <= max;
};

const validarSenha = validar(8, 16);
const validarNomeUsuario = validar(4, 12);

validarSenha(senha);
validarNomeUsuario(nomeUsuario);
```



Memoization

o que é

- "memória" de cálculos anteriores
- armazena os resultados de chamadas anteriores
- melhora o desempenho de funções
 - otimiza algoritmos recursivos
- **desvantagem** consumo de memória

exemplos

```
// função iterativa para calcular fatorial
let fatorial = numero => {
  let resultado = 1;
  for (let i = 1; i <= numero; i++) { resultado *= i; }
  return resultado;
}
```

```
// função recursiva para calcular fatorial
let fatorial = numero => {
  if (numero === 0 || numero === 1) return 1;
  return numero * fatorial(numero - 1);
}
```



Memoization

o que é

- "memória" de cálculos anteriores
- armazena os resultados de chamadas anteriores
- melhora o desempenho de funções
 - otimiza algoritmos recursivos
- **desvantagem** consumo de memória

exemplos

```
// utilizando memoization para calcular fatorial
function fatorialMemo() {
  let cache = {};

  return function fatorial(n) {
    if (n in cache) return cache[n];

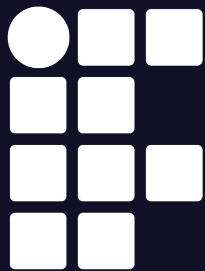
    if (n === 0 || n === 1) return 1;

    cache[n] = n * fatorial(n - 1);
    return cache[n];
  };
}

const fatorial = fatorialMemo();

console.log(fatorial(5)); // 120
console.log(fatorial(5)); // 120 (cache)
```





**INSTITUTO
FEDERAL**
Santa Catarina

Programação Funcional em JavaScript (parte 1)

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas